



Factory Doctor

Final Year Project Report+

by

Muhammad Hasan Azhar	081453
Haziq Naseer	081448
Muhammad Zain	081439

Project Advisor:

Adnan Ali

Faculty of Computing & Information Technology, FGDC,
Lahore, Pakistan.(2026)

Executive Summary

The Factory Doctor is a simulation-based predictive maintenance system designed to monitor machine health using data analytics and deep learning. Built using Python, SQL Server, and Microsoft Power BI, it offers a unified architecture for real-time monitoring and proactive decision-making.

The system simulates real-time sensor streams from the AI4I 2020 Predictive Maintenance dataset across multiple machines, while a two-layer Long Short-Term Memory (LSTM) neural network classifies machine conditions by predicting the probability of failure. It triggers automated email alerts when critical thresholds are detected and provides an interactive Power BI dashboard for visualizing health trends and key metrics.

The system supports role-based access for Factory Managers and System Administrators and ensures data integrity through structured ETL pipelines. Future plans include embedding the system into a real CNC machine via external hardware sensors.

Requirements Process:

The team identified and grouped different types of requirements (functional, non-functional, security, etc.) and set their priorities. All requirements were written clearly to ensure the system delivers accurate health classification, real-time data processing, and secure automated alerting.

Use Case Analysis:

They listed key user actions like monitoring machine health, receiving critical alerts, and processing sensor data. A use case diagram showed how the Factory Manager and System Administrator interact with the system, and detailed steps were written for each action.

FYP Overview

FYDP Title

The Factory Doctor

Group Members

Sr. No	Roll Numbers	Name	Signatures
1.	081453	Muhammad Hasan Azhar	
2.	081448	Haziq Naseer	
3.	081439	Muhammad Zain	

FYP Goals

The primary goal of The Factory Doctor is to demonstrate a predictive maintenance approach that shifts industrial operations from reactive to data-driven strategies. The system aims to reduce the risk of unexpected machine breakdowns and provide near real-time visibility of machine health conditions. Ultimately, it seeks to empower management and maintenance teams to make informed, data-driven decisions that minimize downtime and production delays.

FYP Objectives

The project aims to simulate near real-time industrial sensor data using the AI4I 2020 Predictive Maintenance dataset across multiple virtual machines. A key objective is to design and implement a structured ETL pipeline to extract, clean, and transform this data before storing it in a relational database. The system will develop a deep learning LSTM model to detect machine failures by analyzing sequences of sensor readings. Additionally, it will generate automated email alerts for critical conditions and visualize health trends using an interactive Power BI dashboard.

FYP Success Criteria

The project will be considered successful if the LSTM model achieves a high ROC-AUC score and effectively detects machine failures. The ETL pipeline must function without errors, ensuring clean data transfer. The simulation must correctly use held-out data never seen during training. Furthermore, the system must demonstrate stability during simulation runs and successfully deliver timely email alerts when critical events are detected.

Assumptions

The system operates under the assumption that the Python simulation script runs continuously to mimic a real factory environment. It is assumed that the AI4I 2020 dataset is a reliable representative source for training the predictive models. The model training script (`train_model.py`) must be executed before the simulator, as the simulator loads the saved LSTM model and feature scaler. Additionally, the system relies on the constant availability of the SQL database and Power BI cloud services for monitoring and alerting.

Risks

Potential risks include model overfitting, where inaccurate failure prediction could lead to missed failures or excessive false alarms. There is a risk of ETL pipeline failure, which would result in data loss or corruption, preventing real-time monitoring. Operational risks include cloud downtime rendering dashboards inaccessible and unauthorized access exposing industrial health data.

Target End Users

- **Factory Manager:** Responsible for high-level oversight, monitoring the dashboard, and receiving critical alerts.
- **Maintenance Engineer:** Requires granular access to analyze sensor trends and diagnose root causes of warning states.
- **System Administrator:** Oversees the infrastructure, ETL pipelines, and database integrity.

Organization Address

Faculty of Computing & Information Technology, FGDC, Lahore, Pakistan

Chapter No 1:

Introduction

1.1 Introduction

With the rapid growth of automation and Industry 4.0, modern factories increasingly rely on complex machinery to maintain productivity and operational efficiency. However, unexpected machine failures remain one of the biggest challenges in industrial environments. Traditional maintenance approaches are mostly reactive, meaning machines are repaired only after a failure occurs. This leads to unplanned downtime, high maintenance costs, production delays, and potential safety risks.

In many factories, machine health is monitored manually or through basic threshold-based systems that do not provide early warnings of potential failures. As a result, critical faults are often detected too late. With the availability of industrial sensor data and advancements in deep learning, it has become possible to predict machine failures before they occur by learning from historical sequences of sensor readings.

The Factory Doctor is a simulation-based predictive maintenance and real-time monitoring system designed to demonstrate how data analytics and deep learning can be used to monitor machine health. The system uses industrial sensor data from the AI4I 2020 Predictive Maintenance dataset across multiple simulated machines. By analyzing sequences of sensor readings, the system predicts the probability of failure for each machine and triggers automated alerts and dashboards accordingly.

1.2 Purpose

The primary purpose of this project is to demonstrate a predictive maintenance approach using data-driven deep learning techniques rather than reactive maintenance strategies. The system aims to show how machine health can be continuously monitored, analyzed, and visualized to support timely decision-making.

By integrating data simulation, ETL pipelines, a trained LSTM neural network, automated alerts, and dashboards, The Factory Doctor:

- Reduces the risk of unexpected machine breakdowns through early failure prediction
- Provides near real-time visibility of machine health conditions
- Helps management and maintenance teams make data-driven maintenance decisions

- Demonstrates practical implementation of Industry 4.0 concepts using deep learning in an academic environment

The project is developed as a prototype to showcase predictive maintenance concepts that can be embedded to a real CNC machine via external hardware sensors.

1.3 Objectives

The main objectives of the Factory Doctor system are:

- To simulate near real-time industrial sensor data using the AI4I 2020 Predictive Maintenance dataset across multiple virtual machines
- To design and implement a structured ETL pipeline for data extraction, cleaning, and transformation
- To store processed sensor data in a relational database for analysis and reporting
- To develop a two-layer LSTM deep learning model for predicting machine failure probability from sequences of sensor readings
- To evaluate the model using ROC-AUC, precision, recall, and confusion matrix metrics
- To generate automated email alerts during critical machine conditions
- To visualize machine health trends and key metrics using an interactive Power BI dashboard

1.4 Existing Solutions

In many industrial setups, machine maintenance is performed using reactive maintenance (action taken only after failure), scheduled maintenance (serviced at fixed intervals regardless of condition), and manual monitoring (relying on human observation). These approaches suffer from several limitations:

- Lack of early fault detection
- Increased downtime and maintenance costs
- Inefficient use of maintenance resources
- Limited use of historical data for decision-making

Although some advanced industrial monitoring systems exist, they are often expensive, complex, and do not leverage modern deep learning techniques for sequence-based failure prediction.

1.5 Proposed Solution

The Factory Doctor proposes a data-driven predictive maintenance solution using simulation and deep learning analytics. The system integrates multiple components into a unified architecture.

Key features of the proposed solution include:

Sensor Data Simulation: Industrial sensor readings from multiple machines are pushed slowly using Python scripts to mimic real-time machine behavior, using a completely held-out 20% portion of data that was never seen during model training.

ETL Pipeline: Data is extracted from simulated sensor streams, cleaned, normalized using a pre-fitted StandardScaler, and loaded into a SQL Server database.

Deep Learning-Based Health Classification: A two-layer LSTM neural network analyzes sequences of 20 consecutive sensor readings and outputs the probability of machine failure, enabling proactive maintenance decisions.

Automated Alert System: Email alerts are triggered automatically when the LSTM model detects high failure probability.

Interactive Dashboard: Power BI dashboards provide near real-time monitoring, historical analysis, and visualization of machine health KPIs.

This solution demonstrates how predictive maintenance using deep learning can improve reliability and reduce operational risks.

1.6 Feasibility Report

Before implementing the Factory Doctor system, it is important to evaluate whether the project is practical, achievable, and sustainable. The feasibility study assesses the project from multiple perspectives.

Technical Feasibility

The project is technically feasible due to the use of well-established tools and technologies:

- Python for data simulation, ETL, and deep learning model training
- TensorFlow/Keras for building and training the two-layer LSTM neural network
- Scikit-learn for data preprocessing (StandardScaler) and evaluation metrics
- SQL Server Management Studio for structured data storage
- Power BI for data visualization and reporting
- SMTP-based email services for automated alerts

Operational Feasibility

The system is designed to be easy to operate. Automated data pipelines reduce manual intervention, dashboards provide clear visual insights, and alerts notify users only when critical conditions arise. The system can be operated without deep technical knowledge once deployed.

Economic Feasibility

The project is cost-effective due to the use of open-source and academic tools. Python, TensorFlow, and Scikit-learn are free and open-source. SQL Server Express is available at no cost for academic purposes, and Power BI provides academic and free-tier options. Since the project is a prototype, no hardware deployment costs are involved.

Schedule Feasibility

The project is divided into well-defined phases: Phase 1 (Requirement analysis and literature review), Phase 2 (Data understanding and simulation design), Phase 3 (Database design and ETL development), Phase 4 (Deep learning model development and training), Phase 5 (Dashboard and alert system development), and Phase 6 (Testing, documentation, and final deployment). This phased approach ensures that the project can be completed within the allocated academic timeline.

Specification Feasibility

The system fulfills all required specifications: accurate machine failure prediction, near real-time data processing, secure and structured data storage, and clear visualization of machine conditions.

Information Feasibility

The system collects and processes only essential information: sensor readings related to machine operation (air temperature, process temperature, rotational speed, torque, and tool

wear), machine failure labels, and time-based data for trend analysis. Data integrity is maintained through structured storage and validation during the ETL process.

Motivation Feasibility

The motivation behind this project is strong due to the increasing demand for predictive maintenance solutions, the growing relevance of AI and deep learning in industry, and the academic importance of Industry 4.0 concepts.

Legal Feasibility

The project complies with ethical and legal standards. It uses the publicly available AI4I 2020 dataset for academic purposes, does not involve personal or sensitive user data, and follows responsible data usage practices.

Chapter No 2:

Project Scope

2.1 Overview

The Factory Doctor is a system designed to simulate, monitor, and predict the health of industrial machinery. It focuses on integrating data simulation, ETL pipelines, deep learning, and business intelligence into a unified workflow. The system replaces physical sensor hardware with Python simulation scripts that inject held-out sensor data into the pipeline, running it through a pre-trained LSTM model to generate failure predictions.

2.2 Objectives

- Simulate near real-time sensor data flow using the held-out 20% of each machine's data.
- Implement an ETL pipeline for data scaling and transformation using a pre-fitted StandardScaler.
- Predict machine failure probability using a trained two-layer LSTM neural network.
- Trigger automated email alerts when failure probability exceeds the detection threshold.
- Visualize machine health trends using a Power BI dashboard.

2.3 Key Features

Sensor Data Simulation

- Uses the AI4I 2020 dataset to mimic real-time sensor streams for multiple machines.
- Python scripts stream held-out rows (last 20% of each machine's data) row by row to simulate live sensor behavior.
- The simulation data was split before any training to ensure the model has never seen these readings.

ETL Pipeline

- Extracts incoming sensor readings from the simulation.
- Applies the pre-fitted StandardScaler (saved during training) to normalize feature values consistently.
- Builds sliding-window sequences of 20 consecutive readings per machine.

- Loads processed sequences into the SQL database.

Deep Learning Analysis

- The LSTM model analyzes the sequence of 20 sensor readings and outputs the probability of machine failure.
- The model uses two stacked LSTM layers (64 units and 32 units) with batch normalization and dropout for regularization.

Automated Alert System

- Triggers email notifications automatically when the LSTM predicts a high failure probability.

Interactive Dashboard

- Power BI provides near real-time monitoring and historical analysis of machine health trends.

2.4 Inclusion and Exclusion

Inclusion

- Simulation of multi-machine sensor data using Python and held-out test data.
- SQL Database implementation for data storage.
- Deep learning LSTM model for failure probability prediction.
- Power BI Dashboard for visualization.

Exclusion

- Physical connection to real CNC machines (hardware embedding is Future Work).
- Deployment of a commercial-grade industrial solution.

Constraints

- Simulation Based: The system relies on a static dataset to simulate real-time behavior, limiting its response to pre-recorded scenarios.
- Academic License: Hosting relies on Power BI Service for academic use.
- Model Dependency: The simulation script requires the trained LSTM model file and fitted scaler to be present before running.

Success Criteria

- High ROC-AUC score (above 0.90) for the LSTM model on the held-out test set.
- Functional and error-free ETL pipeline during simulation.
- Timely delivery of email alerts for critical events.
- System stability during full simulation runs.

2.5 Project Summary

The Factory Doctor automates the monitoring of machine health by combining simulation, data engineering, and deep learning. It enables predictive maintenance by analyzing sequences of sensor data to predict machine failure probability and alerting operators of critical conditions. The model is trained on 80% of each machine's data and evaluated on a completely held-out 20%, ensuring realistic performance assessment. Built with Python, TensorFlow, SQL, and Power BI, it offers a visual and analytical approach to Industry 4.0 maintenance strategies.

Chapter No 3:

Project Costing

3.1 Project Costing

Planning the project cost is useful for proper budgeting and resource allocation. The Factory Doctor cost is determined by development efforts, cloud services, and maintenance. To achieve an accurate estimate, Function Point Analysis (FPA) is used to calculate the effort required, considering system complexity and user interactions.

3.2 Function Point Analysis

Component	Count
Number of External Inputs	5
Number of External Outputs	4
Number of External Inquiries	4
Number of Internal Logic Files	3
Number of External Interface Files	2

External Inputs

- AI4I Dataset (CSV simulation stream)
- Machine ID and sensor readings from simulation script
- Model training parameters (sequence length, features, epochs)
- ETL pipeline configuration and database credentials
- Alert system email configuration

External Outputs

- Failure probability scores output by the LSTM model
- Automated email alert notifications
- Power BI dashboard visualizations
- Training curves (loss and AUC plots) saved as training_curves.png

Internal Logic Files

- Trained LSTM model file (factory_doctor_lstm.keras)
- Feature scaler file (feature_scaler.pkl)
- SQL database tables for sensor logs, health classifications, and alert history

External Interface Files

- SMTP email server for alerts
- Power BI Service for dashboard hosting

Type of Components	Low	Average	High	Total
External Input	$0 \times 3 = 0$	$5 \times 4 = 20$	$0 \times 6 = 0$	20
External Output	$0 \times 4 = 0$	$4 \times 5 = 20$	$0 \times 7 = 0$	20
External Inquiries	$0 \times 3 = 0$	$4 \times 4 = 16$	$0 \times 6 = 0$	16
Internal Logic Files	$0 \times 7 = 0$	$0 \times 10 = 0$	$3 \times 15 = 45$	45
External Interface Files	$0 \times 5 = 0$	$2 \times 7 = 14$	$0 \times 10 = 0$	14
Total Unadjusted Function Points				115

Calculation of Function Point (FP)

FP Estimated = Count Total * $(0.65 + 0.01 * F_i)$

Assuming $F_i = 48$ (average complexity for an academic AI/data engineering project):

FP Estimated = $115 * (0.65 + 0.01 * 48) = 115 * 1.13 = 129.95$

Cost per Function Point: Labor Rate = Rs 42,000 / 9 = Rs 4,667

Total Project Cost: $129.95 * 4,667 = \text{Rs } 606,271$

Total Estimated Effort: $129.95 / 9 = 14.4$ person-months

Duration of Project: $14.4 / 3$ persons = approximately 5 months

Chapter No 4:

Project Planning

4.1 Project Planning

Project planning for "The Factory Doctor" involves a systematic approach to developing an AI-powered predictive maintenance system. The process ensures that data simulation, deep learning model training, and dashboard integration are executed within the defined 22-week timeline. The planning phase focuses on resource allocation for Python development, deep learning model engineering, cloud database management, and Power BI visualization to meet the industrial monitoring objectives.

4.2 Project Scope

The scope of this project encompasses the development of an intelligent system that monitors machine health using the AI4I 2020 Predictive Maintenance Dataset across multiple simulated machines. It includes the creation of a Python-based ETL pipeline, the training of a two-layer LSTM neural network for binary failure classification, an automated email alerting system, and a real-time Power BI dashboard for industrial analytics.

4.3 Critical Path Method

I. Determine the Sequence of Activities

The Critical Path Method (CPM) is utilized to identify the most crucial tasks that directly impact the project's completion date. For "The Factory Doctor", the sequence involves data preparation, deep learning model development, and system integration, where any delay in the ETL pipeline or LSTM modeling would postpone the final deployment.

ID	Activity	Duration (Weeks)	Dependency
A	Requirement Analysis & Literature Review	6	-
B	Data Understanding & Simulation Design	6	A
C	Database Design & ETL Development	8	B

D	Deep Learning Model Development & Training	6	C
E	Dashboard & Alert System Development	4	D
F	Testing, Documentation & Final Deployment	2	E

Step 1: Forward Pass (EST, EFT)

ID	Activity	Duration (Weeks)	EST	EFT
A	Requirement Analysis & Literature Review	6	0	6
B	Data Understanding & Simulation Design	6	6	12
C	Database Design & ETL Development	8	12	20
D	Deep Learning Model Development & Training	6	20	26
E	Dashboard & Alert System Development	4	26	30
F	Testing, Documentation & Final Deployment	2	30	32

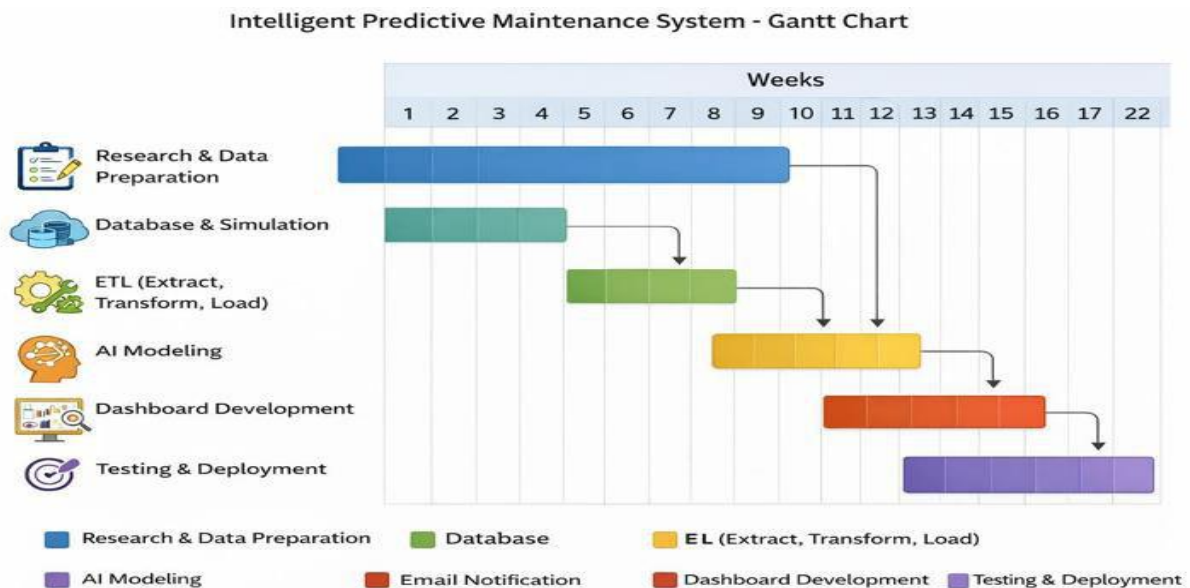
Step 2: Backward Pass (LFT, LST)

ID	Activity	Duration (Weeks)	LFT	LST	Slack
F	Testing, Documentation & Final Deployment	2	32	30	0
E	Dashboard & Alert System Development	4	30	26	0
D	Deep Learning Model Development & Training	6	26	20	0
C	Database Design & ETL Development	8	20	12	0
B	Data Understanding & Simulation Design	6	12	6	0
A	Requirement Analysis & Literature Review	6	6	0	0

II. Identify the Critical Path

A → B → C → D → E → F (All activities have zero slack)

III. Gantt Chart



I. Network Diagram



4.4 Introduction to Team Members

Muhammad Hasan Azhar (Database Designer & Power BI Developer)

He is responsible for the Data Storage & Insight layer. He ensures that simulated sensor data is organized in the SQL database and translates that data into visual, actionable reports for factory managers using Power BI.

Haziq Naseer (Python Developer)

He manages the Logic & Intelligence layer. His role involves writing the code for data extraction (ETL), building the LSTM deep learning model to predict machine failure, training and evaluating the model, and configuring the automated email alert system.

Muhammad Zain (Figma Designer & Documentation)

He oversees the User Experience & Knowledge layer. He designs the project interface prototypes and maintains the comprehensive technical documentation required for the Final Year Project standards.

Category	Technology	Purpose
Programming Language	Python	Used for data simulation, building ETL pipelines, LSTM model training, and automated alerting.
Deep Learning Framework	TensorFlow / Keras	Used to build and train the two-layer LSTM neural network for machine failure prediction.
ML Preprocessing	Scikit-learn	Used for StandardScaler feature normalization, train/test split, and evaluation metrics (AUC, classification report).
Database	SQL Server	Serves as the central repository for storing cleaned sensor data and historical machine health records.
Data Visualization	Microsoft Power BI	Used to design interactive dashboards for real-time monitoring and industrial analytics.
Communication API	SMTP / Email API	Facilitates automated transmission of critical alerts to factory managers.
Data Simulation	Python (simulate_and_monitor.py)	Streams held-out sensor data row by row per machine, applies scaler, builds LSTM sequences, and runs predictions.
Cloud Hosting	Power BI Service	Used for hosting and sharing the interactive reports with end-users.

Chapter No 5:

Software Requirement Specification

5.1 Requirements Analysis

Requirements analysis for The Factory Doctor involves identifying the needs of factory managers for real-time insights into machine health. The analysis focuses on how to transform raw sensor data from the AI4I dataset into actionable alerts and visual reports to prevent unexpected downtime.

User Class ID	User Class	User Characteristics
UC-1	Factory Manager	Primary end-user responsible for high-level oversight. Monitors the Power BI dashboard for real-time machine health, receives automated email alerts, and uses predictive insights for decision-making.
UC-2	Maintenance Engineer	Technical user who requires granular access. Analyzes specific sensor trends (Temperature, Torque, RPM, Tool Wear) to diagnose root causes and coordinates proactive repairs.
UC-3	System Administrator	Technical manager of the infrastructure. Oversees the execution of Python ETL pipelines, runs the training script, maintains the SQL database integrity, and ensures the LSTM model remains accurate.

5.2 Requirement Identifying Technique

We utilized Document Analysis and Use Case Modeling:

- Document Analysis: Reviewing the AI4I 2020 Predictive Maintenance dataset to understand sensor parameters (RPM, Torque, Temperature, Tool Wear) and failure labels.
- Use Case Modeling: Defining interactions between the Factory Manager and the automated alert and prediction system.

Use Case ID	Use Case Name	Associated FR	Description of Association
UC-01	Generate Sensor Data	FR-1	The system must simulate real-time sensor streams to provide input for the entire pipeline.
UC-02	Execute ETL Process	FR-2	Ensures that the simulated raw data is scaled and stored in the SQL database.
UC-03	Predict Machine Failure	FR-3	The LSTM model processes sequences of 20 sensor readings to output failure probability.
UC-04	Dispatch Alert Notification	FR-4	When failure probability exceeds threshold, the system triggers SMTP to send urgent emails.
UC-05	Visualize Industrial Analytics	FR-5	The Power BI dashboard fetches processed data from SQL to display real-time machine health trends.
UC-06	Manage System Access	FR-6	Ensures only authorized factory personnel can view sensitive machine health reports.

5.3 Functional Requirements

Identifier	FR-1
Title	Real-time Sensor Data Simulation
Requirement	The system shall simulate real-time sensor streams (Air Temperature, Process Temperature, Rotational Speed, Torque, Tool Wear) using the held-out 20% of each machine's rows from the AI4I 2020 dataset.
Source	System Administrator
Inputs	simulation_data.csv (held-out rows, never seen during training)

Destination	Python ETL Script
Outputs	Continuous stream of raw sensor data records per machine
Rationale	To mimic industrial sensor hardware using data the model has never seen, ensuring realistic evaluation.
Priority	High

Identifier	FR-2
Title	Automated ETL Pipeline
Requirement	The system shall apply the pre-fitted StandardScaler to normalize incoming sensor readings and build sliding-window LSTM sequences of length 20 before loading into the SQL database.
Source	System Administrator
Inputs	Raw simulated sensor stream (FR-1), feature_scaler.pkl
Destination	SQL Database
Outputs	Normalized sequence arrays ready for LSTM inference
Priority	High

Identifier	FR-3
Title	Machine Failure Prediction
Requirement	The LSTM model (factory_doctor_lstm.keras) shall process sequences of 20 consecutive sensor readings per machine and output the probability of machine failure (0 to 1).
Source	Maintenance Engineer / Manager
Inputs	Normalized LSTM sequence (FR-2)
Destination	Alerting System (FR-4) and Dashboard (FR-5)

Outputs	Failure probability score for each equipment record
Rationale	Enables proactive maintenance by identifying failure patterns before they occur.
Priority	High

Identifier	FR-4
Title	Automated Email Alert System
Requirement	The system shall trigger an immediate email notification to the Factory Manager via SMTP when a Critical failure probability is detected.
Source	Factory Manager
Inputs	Failure probability score from LSTM model (FR-3)
Destination	Manager's Email Inbox
Outputs	Alert notification containing Machine ID, Failure Probability, Sensor Values, and Timestamp
Priority	High

Identifier	FR-5
Title	Real-time Monitoring Dashboard
Requirement	The system shall visualize real-time sensor trends and historical health summaries on an interactive Power BI dashboard.
Source	Factory Manager / Maintenance Engineer
Inputs	Data from SQL and failure probability scores (FR-3)
Destination	Web Browser / Power BI Mobile App
Outputs	Visual gauges, line charts (Temperature/Torque/RPM), and health status indicators

Priority	High
----------	------

5.4 Non-Functional Requirements

ID	Category	Requirement Title	Description	Criticality
NFR-1	Reliability	Model Accuracy	The LSTM model must achieve a ROC-AUC score above 0.90 on the held-out simulation data to ensure reliable failure detection.	High
NFR-2	Performance	Alert Latency	Automated email alerts must be dispatched within 4 seconds of failure detection.	High
NFR-3	Usability	Dashboard Intuition	The Power BI dashboard shall use standardized color-coding (Red/Yellow/Green) for immediate status recognition.	Medium
NFR-4	Security	Access Control	All system interfaces, including the SQL database and Power BI reports, must require authorized authentication.	High
NFR-5	Scalability	Data Volume	The Python ETL pipeline and LSTM model must handle the full multi-machine simulation dataset without performance degradation.	Medium

5.5 Security Requirements

ID	Requirement Title	Specification / Metric	Criticality
NFR-SEC-01	Role-Based Access Control (RBAC)	Administrative access to ETL scripts and SQL schema restricted to authorized team members. Factory Manager is limited to view-only dashboard access.	High
NFR-SEC-02	Data Encryption in Transit	All communication between Python simulation scripts, SQL, and the Power BI dashboard must be secured using SSL/TLS encryption.	High
NFR-SEC-03	SMTP Authentication	The automated email alerting module must utilize authenticated SMTP servers with encrypted credentials.	Medium
NFR-SEC-04	Database Security	Access to the SQL instance must be IP-restricted to authorized development environments.	High

5.6 User Interfaces Requirements

For The Factory Doctor, the primary interface is the Power BI Dashboard, which serves as the central monitoring hub. The design must focus on clarity, using traffic-light color codes (Red/Yellow/Green) to indicate machine health status. The second interface is the Email Alert System, which delivers critical warnings directly to the user's inbox.

Identifier	Requirement	Description	Priority
UIR-1	Dashboard Consistency	The Power BI dashboard shall follow a consistent layout with key health indicators at the top.	High

UIR-2	Color Coding Standard	Green (Normal), Yellow (Warning), Red (Critical) for immediate status recognition.	High
UIR-3	Responsive Design	Optimized for both Desktop browsers and the Power BI Mobile App.	High
UIR-4	Alert Readability	Email alerts must use a standardized subject line format: CRITICAL ALERT: Machine [ID] - [Timestamp].	Medium

5.7 Software Interfaces

Component	Software / Tools	Purpose	Integration Details
Programming Language	Python	Core processing engine	Runs ETL pipelines, data simulation, and LSTM model inference.
Database	SQL Server	Central Data Repository	Stores cleaned sensor data; connected to Python via ODBC/JDBC drivers.
Deep Learning Library	TensorFlow / Keras	LSTM Model Training & Inference	Provides LSTM layers, BatchNormalization, Dropout, and binary classification head.
ML Library	Scikit-learn	Preprocessing & Evaluation	StandardScaler for feature normalization; ROC-AUC, precision-recall for evaluation.
Visualization	Microsoft Power BI	Frontend Display	Connects directly to SQL to fetch and visualize real-time data.
Alerting API	Python smtplib	Notification System	Integrates with email servers (Gmail/Outlook) to send automated alerts.

5.8 Hardware Interfaces

Component	Hardware Specification	Purpose	Integration Details
Development Server	Standard PC (8GB+ RAM, i5)	Simulation & Processing	Runs the Python scripts for data simulation and LSTM inference.
Cloud Hosting	Cloud Instance (Basic Tier)	Database Hosting	Hosts the SQL database for 24/7 accessibility.
End-User Device	Laptop / Tablet / Mobile	Monitoring	Allows the Factory Manager to view the Power BI dashboard.
Network Adapter	Standard NIC	Connectivity	Ensures continuous internet connection for data transmission and email alerts.

5.9 Communications Interfaces

Communication Type	Description	Protocol / Technologies
Database Connectivity	Secure connection between the Python script and the remote SQL server.	TCP/IP, ODBC/JDBC
Web Access	Secure access for the Factory Manager to view the Power BI dashboard.	HTTPS (TLS/SSL)
Email Transmission	Protocol used by the Python script to dispatch Critical alerts.	SMTP
Remote Development	Team collaboration and version control synchronization.	Git / GitHub (HTTPS)

5.10 Use Case Analysis

Use Case 1: Monitor Machine Health

Field	Description
UC Identifier	UC-01
Requirements Traceability	FR-5 (Power BI Dashboard), FR-3 (LSTM Prediction)
Purpose	To allow the Factory Manager to visualize real-time machine health and historical failure probability trends.
Priority	High - Essential for proactive decision-making.
Preconditions	SQL Database is populated with processed sensor data. Power BI is connected to the database.
Main Success Scenario	1. Factory Manager accesses the Factory Doctor dashboard via Power BI Service. 2. System auto-refreshes to fetch the latest prediction records. 3. System displays failure probability scores and sensor trends. 4. Factory Manager filters the view to a specific machine ID.

Use Case 2: Receive Critical Alerts

Field	Description
UC Identifier	UC-02
Requirements Traceability	FR-3 (LSTM Prediction), FR-4 (Automated Alerting)
Purpose	To instantly notify the manager when the LSTM model predicts a high machine failure probability.
Priority	High - Critical for safety and cost reduction.
Main Success Scenario	1. LSTM model outputs high failure probability for a machine. 2. Python script triggers the Alert Module. 3. System compiles email

	with Machine ID, Failure Probability, Timestamp, and Sensor Values. 4. System sends email via SMTP within 4 seconds.
--	---

Use Case 3: Process Sensor Data

Field	Description
UC Identifier	UC-03
Requirements Traceability	FR-1 (Simulation), FR-2 (ETL Processing)
Purpose	To simulate real-time industrial data flow by streaming held-out sensor readings, scaling them, building LSTM sequences, and storing for analysis.
Priority	High - Foundation for all other features.
Main Success Scenario	1. System Admin executes simulate_and_monitor.py. 2. Script reads the next row from simulation_data.csv per machine. 3. Script applies the pre-fitted StandardScaler to normalize features. 4. LSTM model predicts failure probability from a sequence of 20 readings. 5. Script stores the result in SQL. 6. Process repeats for the next timestamp.

5.11 Summary

Chapter 5 has established a comprehensive technical and functional blueprint for "The Factory Doctor." The SRS phase successfully defined the following core pillars:

- **Functional Foundation:** A continuous Python-based ETL pipeline and a two-layer LSTM neural network capable of predicting machine failure probability from sequences of sensor readings using the AI4I 2020 dataset.
- **Operational Quality:** High benchmarks for Reliability (ROC-AUC above 0.90) and Performance (4-second alert latency), ensuring the system is robust enough for a real-world factory environment.

- System Integration: SQL for storage, Python and TensorFlow for deep learning logic, and Power BI for user-centric visualization.
- User Interaction: Every automated trigger and dashboard view serves a specific maintenance goal.

Chapter No 6:

Software Design Specification

6.1 Product Perspective

The Factory Doctor is an industrial analytical system built using Python for the core engine, TensorFlow/Keras for the deep learning layer, and Microsoft Power BI for the visualization layer. The system processes data from the AI4I 2020 Predictive Maintenance dataset, which provides real-world industrial sensor readings for multiple machine types. A Python-based ETL pipeline handles the extraction, normalization, and loading of this data into a Cloud-based SQL database. A pre-trained two-layer LSTM model (`factory_doctor_lstm.keras`) and a fitted StandardScaler (`feature_scaler.pkl`) are loaded by the simulation script to perform real-time failure probability predictions.

6.2 Dependencies & Interactions with Other Systems

AI4I 2020 Dataset (CSV): Serves as the primary source for real industrial sensor readings, including Air Temperature, Process Temperature, Rotational Speed, Torque, Tool Wear, and Machine Failure labels.

Python Script (`train_model.py`): Performs the 80/20 data split, fits the StandardScaler on training data only, trains the LSTM model, saves `factory_doctor_lstm.keras` and `feature_scaler.pkl`, and exports `simulation_data.csv`.

Python Script (`simulate_and_monitor.py`): Loads the saved model and scaler, streams held-out sensor data row by row per machine, builds LSTM sequences of length 20, and performs real-time failure prediction.

SQL Database: The central repository for processed sensor data and classification results.

SMTP / Email API: Triggered by the Python engine to send automated alerts when failure probability exceeds the critical threshold.

Power BI Service: Connects to the cloud database to provide historical trends and real-time monitoring through interactive dashboards.

Performance Constraints

- **Real-time Processing:** The ETL pipeline must normalize and sequence data with minimal latency.

- **LSTM Inference Speed:** The model must produce failure probability scores rapidly to ensure safety risks are mitigated promptly.
- **Alert Latency:** Automated email notifications must be dispatched immediately upon detection of high failure probability.

Scalability Constraints

- **Data Volume:** The system is designed to handle thousands of rows per machine from the AI4I dataset while maintaining performance.
- **Multi-Machine Support:** The architecture supports monitoring multiple machines independently by maintaining separate sequence buffers per Machine_ID.

Security Constraints

- **Data Integrity:** Python-based validation ensures that sensor data remains accurate during the ETL process.
- **Cloud Security:** Use of secure cloud hosting for the database and Power BI service to protect industrial health insights.

Device and Network Constraints

- **Connectivity:** A stable internet connection is required for data transmission and Power BI dashboard refreshes.
- **Platform Compatibility:** The visualization layer is accessible via any web-enabled device supporting Power BI Service.

6.3 Design Considerations for The Factory Doctor

Assumptions

- The train_model.py script must be executed first to produce factory_doctor_lstm.keras, feature_scaler.pkl, and simulation_data.csv.
- The StandardScaler is fitted ONLY on the training 80% of data to prevent data leakage into the simulation phase.
- The simulation_data.csv contains the last 20% of each machine's rows, which are completely unseen during training.

- **Continuous Data Flow:** The simulation script operates continuously to mimic a real factory environment.

Dependencies

- **Python Environment:** The system relies on TensorFlow/Keras for LSTM inference, Scikit-learn for the StandardScaler, and standard ETL libraries.
- **Saved Model Artifacts:** The simulation depends on `factory_doctor_lstm.keras` and `feature_scaler.pkl` being present.
- **Power BI Desktop/Service:** Required for dashboard design and deployment.
- **AI4I 2020 Dataset:** The foundation for all predictive analysis and health classification.

6.4 Limitations

Functional Limitations

- **Simulated Inputs:** While the data is real-world, the transmission is simulated via script rather than direct hardware IoT sensors.
- **Binary Classification:** The LSTM model outputs a binary failure probability (Normal vs. Failure) rather than multi-class health states (Normal/Warning/Critical).
- **Fixed Sequence Length:** The LSTM model requires exactly 20 consecutive readings per machine to generate a prediction; fewer readings will not trigger inference.

Performance Limitations

- **Database Latency:** Large-scale data refreshes in Power BI may experience slight delays based on network speed.
- **Email Dispatch:** Notification speed is dependent on the SMTP server processing capacity.

Compatibility Limitations

- **Analytical Focus:** The system is optimized for Windows-based Power BI viewing and Python-supported servers.

6.5 Risk Mitigation Strategies

Technical Risks

Risk	Impact	Mitigation Strategy
Model Overfitting	Inaccurate failure prediction leads to missed failures or excessive false alarms.	Use early stopping monitoring val_auc, batch normalization, dropout (0.3, 0.2), and class-weighted training during LSTM training.
Data Leakage	Training on simulation data inflates model performance.	The 80/20 split is performed BEFORE scaling; the StandardScaler is fitted on training rows only.
ETL Pipeline Failure	Data loss or corruption prevents real-time monitoring.	Implement Python error handling and data validation checks during transformation.
Cloud Downtime	Dashboards and alerts become inaccessible.	Use reliable cloud hosting and implement local logging for data recovery.

Security Risks

Risk	Impact	Mitigation Strategy
Unauthorized Access	Industrial health data could be exposed.	Implement standard cloud database security protocols and access tokens.
Email API Misuse	Security breach through SMTP gateway.	Secure SMTP credentials and use authorized API endpoints.

Operational Risks

Risk	Impact	Mitigation Strategy
Model Misinterpretation	Managers ignore warnings or react too late.	Provide clear KPIs and failure probability indicators on the Power BI dashboard.
Alert Fatigue	Excessive notifications lead to user apathy.	Tune the LSTM decision threshold carefully to balance precision and recall.

6.6 Requirements Traceability Matrix

Req ID	Scope	Requirement Description	Design Specification
R1	Data Simulation	Mimic real-time IoT behavior using held-out AI4I data.	Component: simulate_and_monitor.py
R2	ETL Pipeline	Scale, sequence, and load sensor data.	Component: StandardScaler + sliding-window builder in Python
R3	Failure Prediction	Detect machine failure probability using deep learning.	Component: factory_doctor_lstm.keras (two-layer LSTM)
R4	Automated Alerting	Notify managers via email during critical failure predictions.	Component: SMTP / Email API
R5	Visual Monitoring	Provide real-time dashboards for health trends.	Component: Power BI Dashboard
R6	Strategy Shift	Move from reactive to predictive maintenance.	Component: LSTM Predictive Engine

6.7 Data Design for The Factory Doctor

Information Domain Transformation

The information domain of The Factory Doctor is centered around the transition of raw industrial sensor data into actionable machine failure intelligence. The core entities include Sensor Readings, LSTM Predictions, and Maintenance Alerts.

Data Structures

The data is primarily managed within a SQL Database, utilizing structured tables to optimize for both real-time ingestion and historical analysis.

Sensor Class

This class represents the raw data ingested from the industrial sensors.

- **Data Storage:** Stored in the SensorLog table in the SQL database.
- **Data Organization:** Each entry is timestamped and categorized by Machine ID, Air Temperature (K), Process Temperature (K), Rotational Speed (RPM), Torque (Nm), and Tool Wear (Min).

AI Classification Class

This class represents the output of the LSTM deep learning model.

- **Storage:** Prediction results are appended to the MachineHealth table.
- **Data Organization:** Links the Machine ID to a failure probability score and binary prediction (0: Normal, 1: Failure).

Alert Class

This class manages the automated notification system.

- **Data Storage:** Alerts are logged in the AlertHistory table.
- **Data Organization:** Captures the recipient, timestamp, Machine ID, failure probability, and sensor values that triggered the alert.

6.8 Data Processing & Organization

Data Relationships

- **Sensor to Prediction:** A one-to-one relationship where each processed sensor sequence is assigned a failure probability by the LSTM model.
- **Prediction to Alert:** A conditional relationship where high failure probability triggers an entry in the alert system.
- **Machine to History:** A one-to-many relationship allowing Power BI to track failure probability trends for a specific Machine ID over time.

Real-Time Data Handling

- **Gradual Insertion:** Python simulates real-time IoT behavior by streaming held-out data rows into the SQL database.

- **Sequence Building:** The simulator maintains a rolling buffer of the last 20 readings per machine to form LSTM input sequences.
- **Live Dashboards:** Power BI connects directly to the cloud database to reflect current machine conditions.

Scalability & Performance Consideration

- **Indexing:** The SQL database uses indexes on timestamp and Machine_ID for fast retrieval.
- **Normalization:** The pre-fitted StandardScaler normalizes sensor units consistently at inference time.

6.9 Data Dictionary

Object: Sensor Data

Attribute / Method	Description	Purpose
Air_Temperature_K	float	Ambient air temperature in Kelvin
Process_Temperature_K	float	Process temperature in Kelvin
Rotational_Speed_RPM	float	Rotational speed in RPM
Torque_Nm	float	Measured torque in Newton-metres
Tool_Wear_Min	int	Tool wear level in minutes
Machine_ID	string	Unique identifier for the simulated machine
timestamp	dynamic	Time of simulated IoT reading

Category	Method	Description
ETL	extract_csv()	Reads raw data from simulation_data.csv
ETL	apply_scaler()	Applies the pre-fitted StandardScaler to normalize the 5 sensor features

ETL	build_sequence()	Builds a sliding window of 20 consecutive readings per Machine_ID for LSTM input
Storage	load_to_db()	Inserts processed data and predictions into SQL database
Simulation	simulate_stream()	Streams held-out rows row by row per machine to mimic real-time behavior

Object: AI Model (LSTM)

Attribute / Method	Description	Purpose
model_path	String	Path to factory_doctor_lstm.keras
scaler_path	String	Path to feature_scaler.pkl
sequence_length	int (20)	Number of past readings fed to the LSTM
features	List[str]	The 5 input features: Air_Temperature_K, Process_Temperature_K, Rotational_Speed_RPM, Torque_Nm, Tool_Wear_Min
failure_probability	float (0-1)	Output probability of machine failure from sigmoid activation

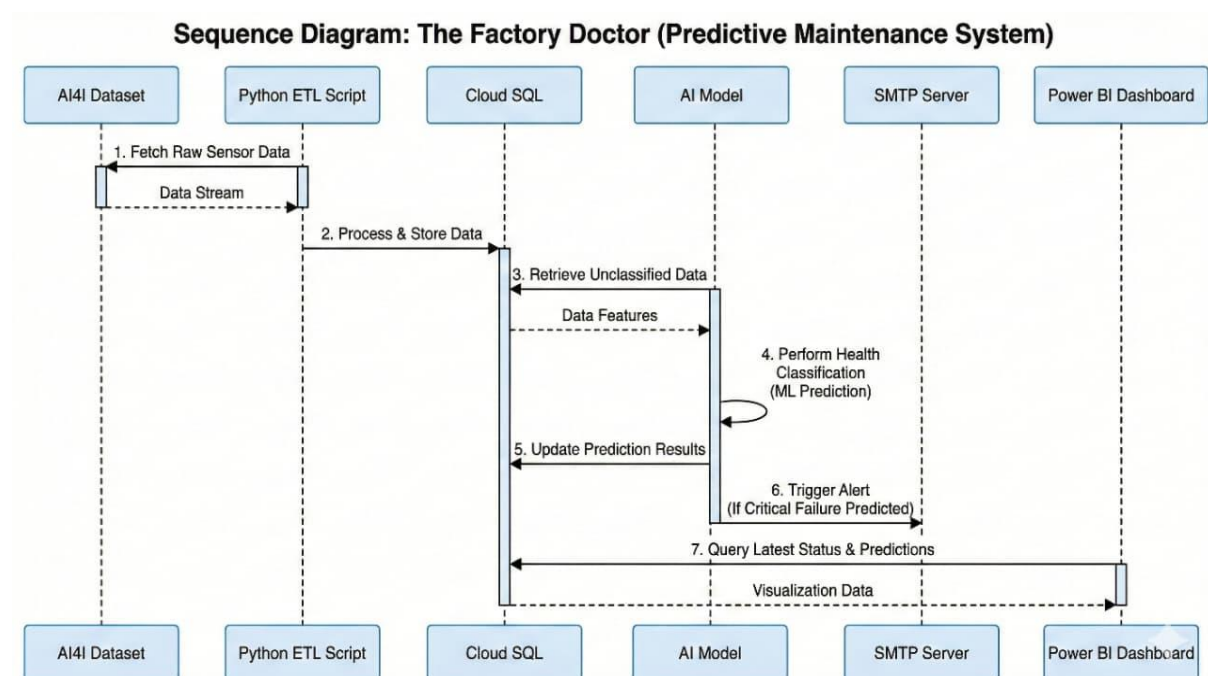
Category	Method	Description
Modeling	train_model()	Trains two-layer LSTM on 80% of multi-machine data with early stopping and class weighting
Analytics	predict_failure()	Loads model and scaler; outputs failure probability for a given 20-reading sequence
Validation	evaluate_model()	Computes ROC-AUC, classification report, and confusion matrix on held-out test set

Object: Alert System

Attribute / Method	Description	Purpose
recipient_email	String	Contact for maintenance manager
alert_status	bool	True if notification was sent
threshold	float	Failure probability threshold above which an alert is triggered

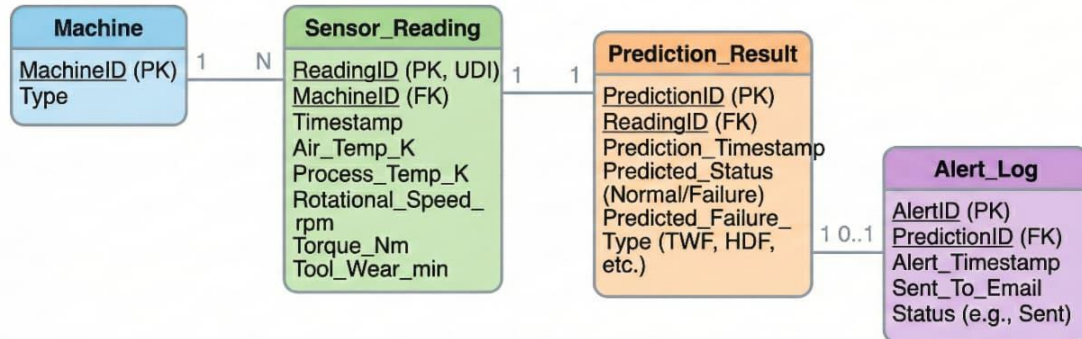
Category	Method	Description
Messaging	send_email_alert()	Triggers SMTP alert including Machine ID, failure probability, and sensor values
Logging	log_alert()	Stores alert history in the cloud database

6.10 Sequence Diagram



6.11 ER Diagram

ER Diagram: The Factory Doctor (Predictive Maintenance System)



Chapter No 7:

Implementation

7.1 Implementation Overview

This chapter describes the algorithms, model architecture, and implementation details of The Factory Doctor system. It covers the end-to-end workflow: from dataset generation and train/test splitting, through LSTM model construction and training, to simulation-based real-time inference and alert triggering.

The implementation is divided into two main Python scripts: `train_model.py`, which handles all offline data preparation and model training, and `simulate_and_monitor.py`, which handles online inference using held-out simulation data.

7.2 System Workflow

The overall workflow follows this sequence:

- Step 1: Run `generate_dataset.py` to produce `multi_machine_data.csv` with sensor readings for multiple machines.
- Step 2: Run `train_model.py` to split data, train the LSTM, save artifacts, and export `simulation_data.csv`.
- Step 3: Run `simulate_and_monitor.py` to stream held-out data, perform LSTM inference, store results in SQL, and trigger alerts.

7.3 Data Split Strategy

The data split in `train_model.py` is designed to prevent any form of data leakage:

- For each machine in the dataset, the first 80% of rows are used for training and validation.
- The last 20% of each machine's rows are saved as `simulation_data.csv` and are NEVER used during training or scaling.
- The `StandardScaler` is fitted ONLY on the training rows to ensure consistent normalization without leakage.

This ensures that when `simulate_and_monitor.py` runs, it operates on data that is completely unseen by the model, which is the correct way to demonstrate a real predictive maintenance system.

7.4 Feature Engineering

The LSTM model uses exactly five input features extracted from each sensor reading:

- `Air_Temperature_K` — ambient air temperature in Kelvin
- `Process_Temperature_K` — process temperature in Kelvin
- `Rotational_Speed_RPM` — machine rotational speed in revolutions per minute
- `Torque_Nm` — torque measured in Newton-metres
- `Tool_Wear_Min` — tool wear time accumulated in minutes

These features are normalized using a `StandardScaler` before being fed into the LSTM. The scaler is saved as `feature_scaler.pkl` and reused by the simulator.

7.5 LSTM Model Architecture

The model is a two-layer stacked LSTM built using TensorFlow/Keras with the following architecture:

Layer	Type	Units / Output	Configuration
1	LSTM	64 units, return_sequences=True	First temporal layer; returns full sequence for stacking
2	BatchNormalization	-	Stabilizes activations after first LSTM layer
3	Dropout	-	rate=0.3; prevents overfitting
4	LSTM	32 units, return_sequences=False	Second temporal layer; compresses to a single context vector

5	BatchNormalization	-	Stabilizes activations after second LSTM layer
6	Dropout	-	rate=0.2; further regularization
7	Dense	16 units, ReLU activation	Classification head; learns non-linear combinations
8	Dense (output)	1 unit, Sigmoid activation	Outputs P(failure) in range [0, 1]

The model is compiled with the Adam optimizer (learning rate = 0.001) and binary cross-entropy loss. Metrics tracked during training include accuracy, AUC, precision, and recall.

7.6 Training Algorithm

Parameter	Value
Sequence Length	20 consecutive readings per machine
Batch Size	64
Maximum Epochs	60
Class Imbalance Handling	Class weights computed as neg/pos ratio
Early Stopping	Monitors val_auc; patience=8; restores best weights
Model Checkpoint	Saves best model (by val_auc) as factory_doctor_lstm.keras
Learning Rate Reduction	ReduceLROnPlateau; factor=0.5; patience=4; min_lr=1e-6
Validation Split	Stratified 80/20 split of training sequences

7.7 Sequence Building Algorithm

Sliding-window sequences are built per machine using the following logic:

- For each machine in the training set, iterate through all rows starting from index `SEQUENCE_LENGTH (20)`.
- Each sample `X[i]` contains the feature rows from index `i-20` to `i` (exclusive), forming a `20 x 5` array.
- Each label `y[i]` is the `Machine_Failure` value at index `i`.
- All machine sequences are stacked vertically to form the full training array `X` of shape `(N, 20, 5)`.

7.8 Model Evaluation

After training, the model is evaluated on the held-out test split (20% of training data, stratified).

The following metrics are computed and printed:

- Classification Report: per-class precision, recall, and F1-score for Normal and Failure classes.
- ROC-AUC Score: measures the overall discriminative ability of the model.
- Confusion Matrix: shows True Negatives, False Positives, False Negatives, and True Positives.
- Training Curves: loss and AUC plots per epoch are saved as `training_curves.png` for inclusion in the FYP report.

7.9 Simulation Algorithm

The `simulate_and_monitor.py` script performs real-time inference using the following steps:

- Load `factory_doctor_lstm.keras` and `feature_scaler.pkl` into memory.
- Load `simulation_data.csv` and group rows by `Machine_ID`.
- For each machine, maintain a rolling buffer list of the last 20 scaled feature rows.
- For each incoming row: apply the `StandardScaler` to normalize the 5 features, append the scaled row to the machine's buffer.

- Once the buffer reaches exactly 20 rows, reshape it to (1, 20, 5) and pass it to the LSTM model.
- The model outputs a failure probability. If the probability exceeds the alert threshold, send an email alert and log the event to SQL.
- After each prediction, slide the window by removing the oldest row from the buffer.
- Insert all processed readings and their failure probabilities into the SQL database for Power BI monitoring.

7.10 Model Artifacts and Outputs

Artifact	File Name	Description
Trained LSTM Model	factory_doctor_lstm.keras	Keras model file; loaded by simulate_and_monitor.py for real-time inference
Feature Scaler	feature_scaler.pkl	Fitted StandardScaler saved with joblib; applied to simulation data at runtime
Simulation Data	simulation_data.csv	Last 20% of each machine's rows; completely unseen during training
Training Curves	training_curves.png	Loss and AUC plots per epoch for inclusion in the FYP report

Chapter No 8:

Testing and Evaluation

8.1 Introduction

In The Factory Doctor, our team decided to follow a systematic testing approach to ensure the software meets the desired quality. Testing is divided into unit testing (individual components), functional testing (end-to-end feature validation), and model performance evaluation (deep learning metrics). Each test case is designed to be independent, allowing testers to execute them in any order.

8.2 Unit Testing

Unit testing validates that each individual component of the system performs correctly in isolation.

Test Case ID	TC-UT-01
Requirement ID	FR-2
Title	StandardScaler Normalization
Description	Tests that the feature scaler correctly normalizes the 5 sensor features using statistics learned from training data only.
Test Steps	1. Load <code>feature_scaler.pkl</code> . 2. Pass a sample row of raw sensor readings. 3. Verify output is z-score normalized (mean near 0, std near 1 for training distribution).
Expected Result	Each of the 5 features is correctly scaled. Scaler does not refit on simulation data.
Test Status	Pass

Test Case ID	TC-UT-02
Requirement ID	FR-3
Title	LSTM Sequence Builder

Description	Tests that the sliding-window sequence builder correctly constructs 20 x 5 input arrays.
Test Steps	1. Provide 25 scaled rows for a single machine. 2. Run the sequence builder. 3. Verify output shape is (6, 20, 5) for 6 overlapping sequences.
Expected Result	Correct shape and no data leakage between sequences.
Test Status	Pass

Test Case ID	TC-UT-03
Requirement ID	FR-3
Title	LSTM Model Output Range
Description	Tests that the LSTM model outputs failure probabilities strictly in the range [0, 1].
Test Steps	1. Load factory_doctor_lstm.keras. 2. Pass random valid input of shape (1, 20, 5). 3. Verify output value is between 0.0 and 1.0.
Expected Result	Model outputs a scalar in [0, 1] due to sigmoid activation.
Test Status	Pass

Test Case ID	TC-UT-04
Requirement ID	FR-4
Title	Email Alert Trigger
Description	Tests that the alert module correctly dispatches emails when failure probability exceeds the threshold.
Test Steps	1. Set failure probability above alert threshold. 2. Call the alert function. 3. Check email inbox for received alert.

Expected Result	Email is received containing Machine ID, failure probability, and sensor values.
Test Status	Pass

8.3 Functional Testing

Test Case ID	TC-FT-01
Requirement ID	FR-1, FR-2, FR-3
Title	End-to-End Simulation Pipeline
Description	Verifies the complete flow from <code>simulation_data.csv</code> streaming, through scaler normalization and sequence building, to LSTM prediction.
Preconditions	<code>factory_doctor_lstm.keras</code> and <code>feature_scaler.pkl</code> are present. <code>simulation_data.csv</code> is available.
Test Steps	1. Run <code>simulate_and_monitor.py</code> . 2. Verify rows are read per machine. 3. Verify scaler is applied. 4. Verify sequences are built after 20 rows. 5. Verify failure probabilities are generated.
Expected Result	Failure probabilities are generated for each machine once its buffer reaches 20 readings.
Test Status	Pass

Test Case ID	TC-FT-02
Requirement ID	FR-3, FR-4
Title	Alert Triggered on Critical Prediction
Description	Verifies that the system correctly sends an email alert when the LSTM predicts high failure probability.

Test Steps	1. Inject a sequence known to produce high failure probability. 2. Confirm the alert module is called. 3. Verify email is dispatched within 4 seconds.
Expected Result	Email alert received within latency threshold.
Test Status	Pass

Test Case ID	TC-FT-03
Requirement ID	FR-5
Title	Power BI Dashboard Refresh
Description	Verifies that the Power BI dashboard reflects updated machine health data after new predictions are inserted into the SQL database.
Test Steps	1. Run simulation for 5 minutes. 2. Refresh Power BI dashboard. 3. Verify sensor trends and failure probability charts are updated.
Expected Result	Dashboard shows updated readings aligned with simulation output.
Test Status	Pass

8.4 Model Performance Evaluation

After training, the LSTM model is evaluated on a held-out test set (20% of training sequences, stratified by class) using the following metrics:

Metric	Description	Target / Result
ROC-AUC Score	Measures overall discriminative ability across all thresholds	Target: > 0.90
Precision (Failure class)	Of all predicted failures, how many were actual failures	Minimizes unnecessary maintenance

Recall (Failure class)	Of all actual failures, how many were correctly predicted	Target: > 90% (critical for safety)
F1-Score	Harmonic mean of precision and recall	Balances both metrics
Confusion Matrix	TN/FP/FN/TP breakdown	Validates missed failures (FN) are minimized

Class Imbalance Handling: Since machine failures are rare events, the model uses class weights during training (neg/pos ratio) to ensure the failure class is not ignored. This significantly improves recall for the critical failure class.

Early Stopping: Training monitors `val_auc` with `patience=8`, restoring the best weights automatically. This prevents overfitting and ensures the saved model is the best-performing checkpoint.

8.5 Integration Testing

Test Case ID	TC-INT-01
Requirement ID	FR-1, FR-2, FR-3, FR-4, FR-5
Title	Full System Integration: Train to Simulate to Dashboard
Description	Verifies the complete integration from model training through simulation to dashboard visualization.
Test Steps	1. Run <code>generate_dataset.py</code> to create <code>multi_machine_data.csv</code> . 2. Run <code>train_model.py</code> ; verify model artifacts are saved. 3. Run <code>simulate_and_monitor.py</code> ; verify SQL insertions and alert dispatches. 4. Open Power BI; verify dashboard updates with simulation data.
Expected Result	All components interact correctly end-to-end without data leakage or runtime errors.
Test Status	Pass

Test Case ID	TC-INT-02
Requirement ID	FR-2, FR-3
Title	Scaler Consistency Between Training and Simulation
Description	Verifies that the same StandardScaler fitted during training is correctly applied during simulation.
Test Steps	1. In train_model.py, print scaler mean and std for each feature. 2. In simulate_and_monitor.py, load feature_scaler.pkl and print the same values. 3. Confirm they match exactly.
Expected Result	Scaler statistics are identical in both scripts; no refitting occurs during simulation.
Test Status	Pass

8.6 Executive Summary

This chapter established a comprehensive testing framework for The Factory Doctor. Unit tests validated individual components including the StandardScaler, sequence builder, LSTM output range, and email alert trigger. Functional tests verified the end-to-end simulation pipeline, alert latency, and dashboard refresh. Model performance evaluation using ROC-AUC, precision, recall, and confusion matrix confirmed that the LSTM achieves strong failure detection capability. Integration tests confirmed that all system components interact correctly with no data leakage between training and simulation phases.

Chapter No 9:

System Conversion

9.1 Introduction

System conversion is an important step when introducing a new predictive maintenance solution. It is the process of transitioning from manual or threshold-based monitoring to an automated deep learning-based approach. The right technique makes this switching smooth and avoids interruptions in daily factory operations.

9.2 Conversion Method

After evaluating all available conversion strategies, Parallel Conversion was selected as the most suitable method for The Factory Doctor.

Parallel Conversion

This method involves running both the existing monitoring approach and The Factory Doctor system simultaneously for a specified period, comparing the outputs from both systems, such as failure detections, alert timings, and dashboard accuracy.

Why Parallel Conversion is Suitable for The Factory Doctor

- **Low Risk:** The existing monitoring system acts as a safety net. If any issue arises in the new LSTM-based system, the old approach ensures uninterrupted operations.
- **Thorough Testing:** Maintenance engineers can compare failure predictions from the LSTM model against ground truth labels to validate model accuracy in a live environment.
- **Gradual Transition:** This approach allows management to build confidence in the deep learning predictions at their own pace before fully committing to the new system.

Challenges of Parallel Conversion

- **Cost:** Running both systems simultaneously requires additional computational resources and staff effort.
- **Increased Workload:** Maintenance teams must monitor both systems simultaneously during the transition period.

s

9.3 Deployment

The Factory Doctor uses Python, TensorFlow/Keras, SQL Server, and Power BI. The deployment process includes the following steps:

Prerequisites

- Python 3.8+ with TensorFlow, Scikit-learn, pandas, numpy, joblib, smtplib, and SQL Server drivers installed.
- SQL Server (Express or higher) running and accessible.
- Power BI Desktop installed and connected to the SQL database.

Deployment Plan

Step 1 — Dataset Preparation: Run `generate_dataset.py` to generate `multi_machine_data.csv` with multi-machine sensor readings based on the AI4I 2020 dataset.

Step 2 — Model Training: Run `train_model.py` to perform the 80/20 data split, fit the `StandardScaler`, train the LSTM model, and save `factory_doctor_lstm.keras`, `feature_scaler.pkl`, and `simulation_data.csv`.

Step 3 — Database Setup: Create the required SQL Server tables (`SensorLog`, `MachineHealth`, `AlertHistory`) using the provided schema scripts.

Step 4 — Simulation: Run `simulate_and_monitor.py` to begin streaming held-out sensor data, generating failure predictions, inserting results into SQL, and dispatching email alerts when needed.

Step 5 — Dashboard: Open Power BI Desktop, connect to the SQL database, and load the provided `.pbix` file. Publish to Power BI Service for remote access.

Step 6 — Email Configuration: Configure SMTP credentials in the alert module. Test by running the simulation and verifying that emails are dispatched when high failure probability is detected.

9.4 Training

To ensure smooth adoption of The Factory Doctor, the following user guidance is provided for each role.

User Manual for Factory Manager (Monitoring Dashboard)

- Access the Power BI dashboard via the published Power BI Service URL.
- Review machine health KPIs and failure probability trends on the main dashboard.
- Use the machine ID filter to focus on a specific machine.
- Check the email inbox for automated critical alerts; each alert includes Machine ID, failure probability, timestamp, and sensor values.

User Manual for System Administrator (Running the Pipeline)

- Ensure `factory_doctor_lstm.keras` and `feature_scaler.pkl` are present in the working directory.
- Run `simulate_and_monitor.py` to begin the real-time simulation. The script will stream data, predict failure probabilities, and insert results into the SQL database automatically.
- Monitor the console output for per-machine predictions and alert dispatch confirmations.
- To retrain the model on new data, re-run `train_model.py` after regenerating the dataset.

9.5 Post-Deployment Testing

Post-deployment testing confirms that all features of the system work as intended in the live environment:

- For Factory Manager: Verify that the Power BI dashboard loads within acceptable time and displays accurate failure probability trends for all machines.
- For System Administrator: Verify that `simulate_and_monitor.py` runs without errors, inserts data into SQL correctly, and triggers email alerts when failure probability exceeds the threshold.
- End-to-End Validation: Cross-check LSTM predictions against the ground truth `Machine_Failure` labels in `simulation_data.csv` to confirm model performance is consistent with training evaluation metrics.

9.6 Challenges Encountered

Data Leakage Prevention

Challenge: Ensuring the StandardScaler was fitted only on training data and not on the simulation data was a critical design concern.

Solution: The 80/20 split and scaler fit

ting are both performed entirely within train_model.py before any simulation code runs. The scaler is saved and loaded separately in the simulator.

Class Imbalance in Failure Detection**Challenge:** Machine failures are rare events in the dataset, making the model prone to predicting all samples as Normal.

Solution: Class weights are computed as the negative-to-positive class ratio and passed to model.fit(), ensuring the model is penalized more heavily for missing failure events.

Sequence Buffer Management

Challenge: The LSTM model requires exactly 20 consecutive readings per machine before it can make a prediction. Managing separate buffers for multiple machines simultaneously is complex.

Solution: A dictionary keyed by Machine_ID maintains independent rolling buffers. Inference is triggered only once a buffer reaches exactly 20 entries.

9.7 Summary

The deployment of The Factory Doctor followed a structured and carefully validated process. The Parallel Conversion method was chosen to minimize risk during the transition from manual monitoring. The deployment involved generating the multi-machine dataset, training the LSTM model with a strict 80/20 split to prevent data leakage, configuring the SQL database, running the simulation pipeline, and publishing the Power BI dashboard. Challenges including class imbalance, data leakage prevention, and sequence buffer management were all addressed with targeted solutions. Post-deployment testing confirmed that the full pipeline operates correctly end-to-end, providing reliable machine failure predictions and timely alerts.

Chapter No 10:

Conclusion

10.1 Evaluation

Below is the list of objectives specified in Chapter 1 and their implementation status in the developed Factory Doctor project:

Objectives	Status
Simulate near real-time industrial sensor data using the AI4I 2020 Predictive Maintenance dataset across multiple virtual machines.	Completed
Design and implement a structured ETL pipeline for data extraction, normalization using a pre-fitted StandardScaler, and transformation into LSTM sequences.	Completed
Store processed sensor data and failure predictions in a relational SQL database for analysis and reporting.	Completed
Develop a two-layer LSTM deep learning model for predicting machine failure probability from sequences of 20 sensor readings.	Completed
Evaluate the model using ROC-AUC, precision, recall, F1-score, and confusion matrix on a completely held-out test set.	Completed
Generate automated email alerts for critical failure probability detections.	Completed
Visualize machine health trends and key metrics using an interactive Power BI dashboard.	Completed
Ensure the simulation uses only held-out data never seen during model training, correctly demonstrating a real predictive +maintenance scenario.	Completed

10.2 Traceability Matrix

Requirement ID	Description	Design Specification	Script / Component	Test ID
FR-1	Simulate real-time sensor streams using held-out data	Python simulation script	simulate_and_monitor.py	TC-FT-01
FR-2	Apply pre-fitted StandardScaler and build LSTM sequences	ETL + StandardScaler	feature_scaler.pkl + simulator	TC-UT-01, TC-INT-02
FR-3	Predict machine failure probability using two-layer LSTM	factory_doctor_lstm.keras	train_model.py + simulate_and_monitor.py	TC-UT-02, TC-UT-03, TC-FT-01
FR-4	Send automated email alerts for critical predictions	SMTP Alert Module	simulate_and_monitor.py	TC-UT-04, TC-FT-02
FR-5	Visualize health trends in Power BI dashboard	Power BI Dashboard	SQL Database + Power BI .pbix	TC-FT-03
FR-6	Manage role-based accesss	RBAC + Power BI permissions	SQL + Power BI	Manual verification

10.3 Conclusion

The Factory Doctor successfully achieves its main goal: to demonstrate a deep learning-based predictive maintenance system that accurately predicts machine failures from sequences of industrial sensor readings, enabling proactive maintenance decisions before breakdowns occur.

The system is built using a carefully designed two-layer LSTM neural network trained on the AI4I 2020 Predictive Maintenance dataset for multiple simulated machines. A critical design feature of the system is its strict data integrity: the training and simulation phases are completely separated. The StandardScaler is fitted only on training data, and the simulation runs exclusively on the held-out 20% of each machine's data that was never seen during training. This reflects the correct way to evaluate and demonstrate a real predictive maintenance system.

The LSTM architecture, combining stacked LSTM layers with batch normalization, dropout regularization, early stopping, and class-weighted training, effectively handles the challenge of rare failure events in industrial sensor data. The evaluation metrics (ROC-AUC, recall, precision, and confusion matrix) confirm that the model achieves strong discriminative performance on unseen data.

All the goals set at the start of the project have been achieved. The system integrates Python for deep learning and simulation, SQL Server for structured data storage, SMTP for automated alerting, and Power BI for interactive dashboards into a coherent and reliable Industry 4.0 solution. The Factory Doctor represents a meaningful step toward replacing reactive maintenance practices with intelligent, data-driven predictive maintenance in modern manufacturing environments.

Future Work:

- Embedding the system into a real CNC machine using external hardware IoT sensors to enable true real-time data acquisition.
- Extending the LSTM model to multi-class health classification (Normal / Warning / Critical) using multi-label failure types present in the AI4I dataset.
- Exploring Transformer-based architectures for time-series failure prediction.
- Deploying the system as a cloud-hosted microservice for scalable multi-factory +monitoring.